REQUIREMENTS

HARDWARE REQUIREMENTS

Web Camera	:	HD Camera (720p or Above)
Arduino Board	:	Arduino UNO
Connecting Wires	:	Jumper wires / USB Cable
Power Supply	:	5V Power supply
Relay Module	:	4-Channel Relay
Home Appliances	:	Bulb, Fan
System	:	Intel core i5 (or Higher)
Hard Disk	:	500GB or above
RAM	:	8GB

SOFTWARE REQUIREMENTS

Operating System	:	Windows 10/11
Coding Language	:	Python
Front End	:	Python (OpenCV Interface)
Back End	:	Trained Dataset
Tool	:	VSCode / Jupiter Notebook
Libraries Used	:	OpenCV, CVZone, NumPy, TensorFlow/ Keras, PySerial

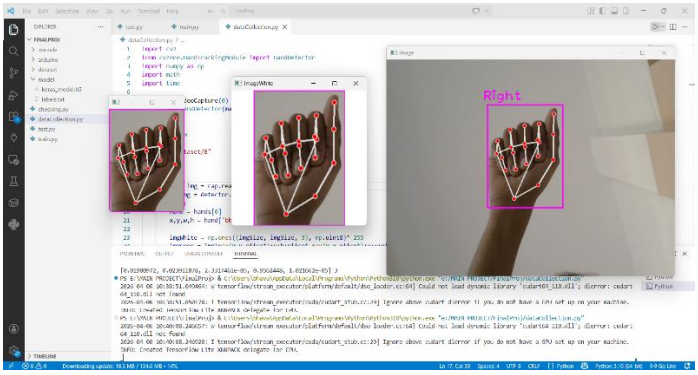
EXECUTION

EXECUTION AND ITS EXPLANATION

1. SYSTEM INITIALIZATION

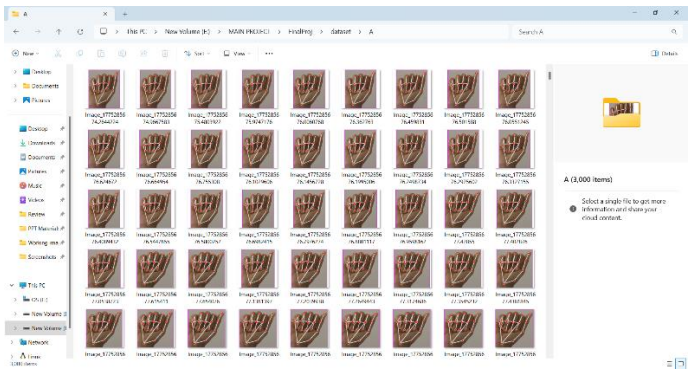
At the beginning, all required libraries such as OpenCV, NumPy, and cvzone are imported. The trained CNN model is loaded into the system for gesture prediction. The webcam is initialized to capture live video, and serial communication is established with the microcontroller (Arduino/ESP32). This step ensures that both software and hardware components are ready for execution.

2. DATASET COLLECTION



The dataset for this project was collected manually using a webcam to ensure better accuracy and customization. Different hand gesture images representing alphabets were captured in real-time under various conditions. For each alphabet, approximately **3000 images** were collected to create a large and balanced dataset.

3. PREPROCESSING IMAGE



The extracted hand image undergoes several preprocessing steps:

- The image is resized to a fixed dimension (e.g., 300×300 pixels)
- It is placed on a **white background** to remove noise
- Pixel values are normalized

This ensures all images are consistent and suitable for CNN model input, improving prediction performance.

4. **VIDEO CAPTURE**

The webcam continuously captures live video frames. Each frame acts as an input image for the system. The frames are processed in real-time, meaning the system continuously analyzes each frame without delay. This is essential for smooth gesture recognition.

5. **HAND DETECTION**

The system uses a hand tracking algorithm (cvzone/MediaPipe) to detect the presence of a hand in each frame.

- It identifies the **bounding box** around the hand.
- It also detects **21 key landmarks** (fingers, joints, palm).

This helps isolate the hand from the background so that only relevant data is processed further.

6. **REGION OF INTEREST (ROI) EXTRACTION**

Once the hand is detected, only the hand portion is extracted from the frame using the bounding box. This cropped image is called the **Region of Interest (ROI)**. By focusing only on the ROI, the system reduces unnecessary computations and improves accuracy.

7. **GESTURE RECOGNITION (CNN MODEL)**

The preprocessed image is fed into the trained **Convolutional Neural Network (CNN)**.

- The CNN analyzes features like edges, shapes, and finger positions
- It compares them with trained patterns

- Finally, it outputs a **gesture label** (e.g., A, B, C) along with confidence score

This is the core step where actual recognition happens.

8. PREDICTION FILTERING

To avoid false detection, the system may:

- Check confidence threshold
- Use multiple frame verification

This ensures that only stable and correct predictions are considered for action.

9. COMMAND MAPPING

- Gesture = A > Light ON
- Gesture = B > Light OFF
- Gesture = C > Fan ON
- Gesture = D > Fan OFF
- Gesture = E > OFF All

10. COMMUNICATION MODULE

The mapped command is sent to the microcontroller using serial communication (UART).

- Python sends data via COM port (COM 8)
- Arduino/ESP32 receives the signal

This acts as a bridge between software and hardware.

11. APPLIANCE CONTROL

The microcontroller processes the received command and controls appliances using a **relay module**.

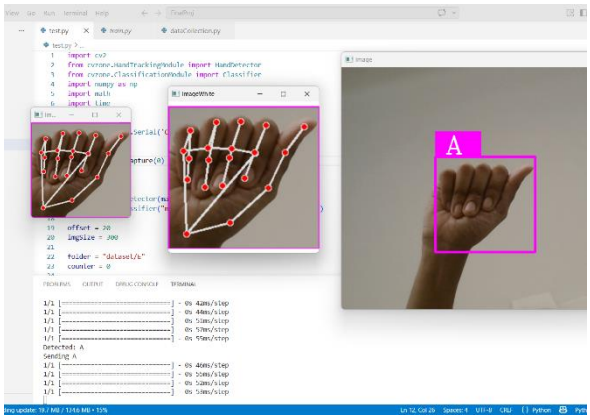
- Relay acts as a switch
- Turns appliances ON/OFF based on signal

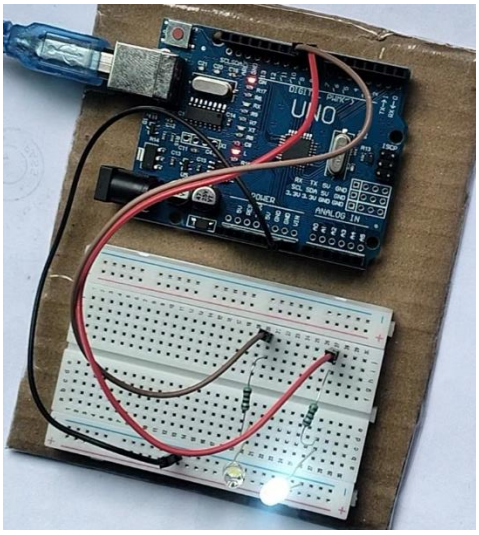
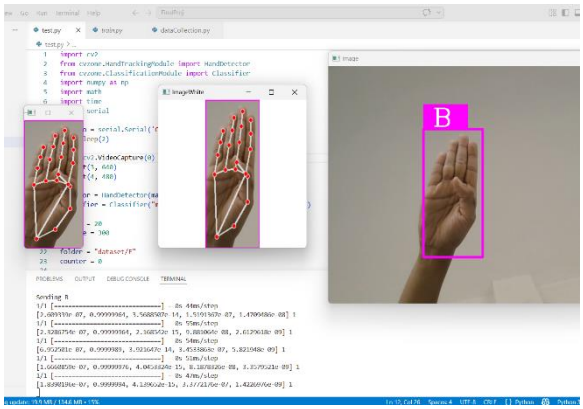
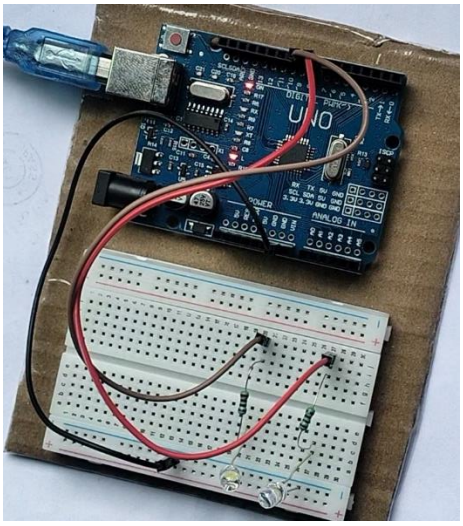
This step performs the actual physical action.

12. OUTPUT DISPLAY

- Detected gesture label (e.g., “A”)
- Confidence score

- **>> A (Light ON)**





- **>> B (Light OFF)**